Collider Research Plan v2: Semantic Graph & Agent Orchestration

Context: Local Xeon/RTX 3060 Workstation | PIKE-RAG Methodology | Recursive Pydantic-Graph

Part 1: Executive Summary & Vision

The Shift: Moving from a purely manual execution graph to a Semantic, Agent-Constructed Universe.

We are not just executing graphs; we are using local GPU power to reason about Pydantic I/O contracts, using retrieved Containers as "Tools" for DeepAgents to solve complex problems recursively.

The "Iterative Refinement" Loop:

1. **Abstract Intent:** User (or Agent) defines a Container with only high-level I/O (e.g., "Analyze Sales Data").
2. **Semantic Retrieval (RAG):** System embeds this I/O signature and searches the vector store for existing Definitions/Tools that match.
3. **Agent Reasoning (PIKE-RAG):** The DeepAgent selects relevant "Knowledge Atoms" (Definitions) and constructs the "Rationale" (the dependency graph/Links).
4. **Concretization:** The abstract Container is populated with a concrete Subgraph.

Part 2: Detailed Research Prompts

1. Semantic Schema Embedding (The "Brain")

Objective: Utilize the RTX 3060 to perform vector operations on Pydantic field definitions, not just docstrings.

- **Challenge:** How to embed a Pydantic `Field(..., gt=0)` so it matches mathematically with a requirement for `PositiveInt`?
- **Approach:**
  - Serialize Pydantic schemas to JSON-Schema or highly descriptive text.
  - Use a code-optimized embedding model (e.g., `jina-embeddings-v2-base-code` or `starcoder`) locally.
  - **Research Question:** Can we train a lightweight adapter (LoRA) to align generic text embeddings with Pydantic strictness?

2. PIKE-RAG Integration for Graph Building

Objective: Apply "Knowledge Atomizing" and "Rationale Augmentation" to the Collider graph.

- **Mapping:**
  - *Knowledge Atom:* A single Definition (Atomic Tool) or Container (Composite).
  - *Rationale:* The specific chain of Links (edges) created to solve a prompt.
- **Algorithm:**
  - **Decomposition:** Agent breaks a user request into sub-tasks (Abstract Containers).
  - **Retrieval:** Agent queries the local vector DB for Atoms matching the sub-tasks.
  - **Reasoning:** Agent verifies I/O compatibility (Type Intersection) between the retrieved Atoms.

3. Pydantic-Graph Integration (Execution & Tool Use)

Objective: Use pydantic-graph as the execution engine for the "Rationale".

- **Recursive Tools:** If an Agent needs to "Analyze Data," and it retrieves a Composite Container, that entire Container (and its internal graph) must become a *single callable tool* for the Agent.
- **Dynamic Injection:** How to inject a retrieved Subgraph into the running Agent's context without restarting the runtime.

4. The "Abstract-to-Concrete" Lifecycle

Objective: Define the lifecycle states of a Container.

- **States:**
  - Draft/Abstract: Only Description + Desired Output defined.
  - Schema-Locked: Input/Output Pydantic Models defined.
  - Linked: Dependencies (Links) added, but not fully mapped.
  - Executable: All Links valid, Topology confirmed.
- **Agent Task:** The "Collider Pilot" agent's goal is to move Containers from *Draft* to *Executable*.

Part 3: Hardware Optimization Strategy

- **Vector Store:** Use Qdrant or Chroma running locally (Docker).
- **Embedding Pipeline:** Dedicated GPU stream for re-indexing Containers whenever their Pydantic schema changes (Observer pattern).
- **LLM Inference:** Serve Mistral-Nemo-12B or Llama-3-8B-Instruct (GGUF/EXL2) via llama-cpp-python or vLLM for the "Pilot" agent.

Success Criteria

✅ Schema Vectorization: Ability to find a tool by its Input Type.

✅ Agent Tool-Use: DeepAgent calling a "Sub-Graph" as a function.

✅ PIKE-RAG Logic: Traceable reasoning logs showing why an Agent linked Container A to Container B.

✅ Recursive Execution: A graph that runs another graph as a node.